UNIVERSITY OF MINES AND TECHNOLOGY, TARKWA

FACULTY OF COMPUTING AND MATHEMATICAL SCIENCES

DEPARTMENT OF CYBERSECURITY AND INFORMATION SYSTEMS

REPORT ON

PASSIVE RECONNAISSANCE AND OSINT GATHERING FOR A SIMULATED TARGET ORGANIZATION

RED TEAM - End-of-Semester Project Report

BY

AMEYAW SARPONG BISMARK

Index Number: FCM.41.018.044.23

Course Code: CY376 - Network Monitoring, Security and Auditing

GitHub Repository: https://github.com/ameyawbismark/Passive-Reconnaissance-and-OSINT-Gathering-for-a-Simulated-Target-Organization.git

Monday, 3rd August 2026

Table of Contents

Abstract

This project undertakes a passive reconnaissance and open-source intelligence (OSINT) gathering exercise against a simulated target organization, Meridian Cove Logistics (MCL), a fictitious freight-forwarding company created specifically for this engagement. Using a fictitious target rather than a real, non-consenting organization keeps the entire exercise inside ethical and legal bounds while still producing genuine, verifiable technical evidence, since the target infrastructure is owned and controlled by the author.

A complete public-facing footprint was designed and deployed for MCL, comprising a corporate website, a non-functional customer portal page, standard crawler-control files (robots.txt and sitemap.xml), and a downloadable company brochure, all hosted on Netlify's free static-hosting tier. Using this live footprint as the assessment scope, a series of passive reconnaissance techniques were applied without any direct interaction with, or scanning of, systems belonging to a real third party: DNS resolution and reverse-IP/ASN lookups, HTTP response header analysis, automated and manually-verified subdomain enumeration, search-engine indexing checks, employee and email-naming-convention enumeration, and PDF metadata extraction.

All findings presented in this report were derived from genuine tool output rather than fabricated data. Of particular note, the automated subdomain enumeration exercise (using theHarvester) surfaced a methodologically significant false positive: candidate subdomains generated through wordlist guessing resolved successfully at the DNS level, owing to wildcard DNS on the underlying shared hosting platform, yet failed TLS certificate validation when queried over HTTPS. This demonstrates that DNS resolution alone is an insufficient signal of live, provisioned infrastructure when a target is hosted on modern platform-as-a-service (PaaS) infrastructure. Separately, metadata embedded in a publicly downloadable PDF brochure independently corroborated personnel names and technology-stack details that had already been gathered through unrelated OSINT techniques, illustrating how seemingly innocuous published documents can reinforce an attacker's confidence in previously-collected intelligence.

The results demonstrate both the practical value and the genuine limitations of passive reconnaissance against a small, cloud-hosted organization, and inform a concrete set of recommendations, including document metadata hygiene, HTTP security header hardening, and mitigation of predictable employee email-naming conventions, presented in the Analysis and Recommendations section of this report.

1. Introduction

1.1 Background

Passive reconnaissance is the practice of gathering information about a target organization exclusively from publicly available sources, without sending traffic directly to, or otherwise interacting with, the target's systems in a way that could be logged, detected, or attributed. It sits at the very start of the intrusion lifecycle described in frameworks such as MITRE ATT&CK, under the Reconnaissance tactic (TA0043), and is typically the first phase a real adversary undertakes before any active engagement with a target's infrastructure begins.

Open-source intelligence (OSINT) refers to the broader discipline of collecting and analyzing information from publicly accessible sources: DNS records, search engine caches, social media, job postings, published documents, and certificate transparency logs, among others. In a security assessment context, passive reconnaissance and OSINT gathering serve a dual purpose. From an attacker's perspective, they build a profile of the target's technology stack, personnel, and exposed services at effectively zero risk of detection. From a defender's perspective, the same techniques reveal exactly what an attacker could learn about the organization before ever touching a production system, which is precisely the value this project sets out to demonstrate.

1.2 Problem Statement

Organizations frequently and unknowingly expose meaningful information about their internal operations through channels that are not obviously sensitive: DNS configuration, job advertisements that list specific software and hardware in use, staff directories with predictable email-naming conventions, and metadata embedded by default in published office documents. None of this information requires any illegal or even mildly invasive activity to obtain, yet in aggregate it can materially lower the cost of a subsequent social-engineering or targeted attack, for example by allowing an attacker to construct a convincing spoofed internal email using a real employee's name, correct email format, and accurately-guessed software environment.

1.3 Objectives

This project set out to:

- Design and deploy a realistic, fully public digital footprint for a simulated organization, built specifically to be safe and legal to test against.
- Apply a structured set of passive reconnaissance and OSINT techniques against that footprint, using freely available, industry-standard tools.
- Capture genuine, unmodified tool output as evidence, rather than illustrative or invented findings.
- Critically evaluate the reliability of automated tool output, including identifying and explaining any false positives encountered.

- Translate the resulting findings into concrete, actionable security recommendations for the (simulated) organization.

1.4 Scope

This is a Red Team project restricted entirely to passive techniques: no port scanning, exploitation, credential testing, or any form of active probing was performed against any system. The target, Meridian Cove Logistics, is an entirely fictitious organization invented for this course, with its public web presence built, deployed, and owned by the author on Netlify's free hosting tier for the sole purpose of this assessment. No real, non-consenting third-party organization, domain, or individual was tested, scanned, or referenced at any point in this project, in line with the course's requirement that Red Team work remain confined to instructor-approved, author-controlled environments.

One practical constraint shaped the scope: due to budget limitations, a dedicated custom domain (e.g. meridiancovelogistics.com) was not registered. The organization's entire public footprint instead resides under Netlify's free *.netlify.app shared subdomain. This is disclosed transparently throughout the report, and its effect on certain findings, particularly subdomain enumeration, is discussed explicitly in Section 6.

1.5 Report Structure

Section 2 reviews the frameworks, standards, and tools that shaped the approach taken. Section 3 describes the lab setup and the methodology followed. Section 4 details the simulated organization that was built and deployed as the assessment target. Section 5 presents the results of each reconnaissance technique applied, with captured tool output. Section 6 analyzes these findings and proposes recommendations. Section 7 concludes the report.

2. Literature and Tooling Review

2.1 Frameworks and Standards

The MITRE ATT&CK framework [1] provided the primary conceptual structure for this project. Its Reconnaissance tactic (TA0043) enumerates the specific techniques adversaries use to gather information prior to an intrusion, several of which map directly onto the work carried out here: Search Open Websites/Domains (T1593), which covers search-engine and public-record based discovery; Search Open Technical Databases (T1596), which covers DNS, certificate transparency, and WHOIS-style lookups; and Gather Victim Identity Information (T1589), which covers the enumeration of employee names, roles, and email addresses. Framing the project's methodology around these named techniques, rather than an ad-hoc list of tools, kept the work aligned with how real threat intelligence and red-team engagements are typically documented and reported.

NIST Special Publication 800-115, the Technical Guide to Information Security Testing and Assessment [2], was also consulted for its treatment of the planning and information-gathering phases of a security assessment, and for its general distinction between passive review techniques and active testing techniques, which this project deliberately restricted itself to the former.

The OSINT Framework [3], a community-maintained directory of categorized open-source intelligence tools and resources, was used during planning to identify which categories of passive technique (DNS/network intelligence, document metadata, email/employee enumeration, and search-engine-based discovery) would be both feasible and instructive to demonstrate within the time and budget available for this project.

2.2 Tools Reviewed and Used

The following tools were selected for their availability at no cost, their common use in real red-team and OSINT engagements, and their suitability for a small, single-domain target:

- nslookup / DNS resolution — used to resolve hostnames to IP addresses and to test whether candidate subdomains actually exist at the DNS layer. A local resolver issue encountered during testing was worked around by explicitly querying Google's public resolver (8.8.8.8), which also served as a useful illustration of resolver-dependent behavior.
- ipinfo.io reverse-IP/ASN lookup — used to attribute resolved IP addresses to their owning Autonomous System and hosting provider, revealing the underlying cloud infrastructure behind the target's PaaS hosting.
- curl — used both for HTTP response header inspection (via the -I flag) and, critically, to distinguish DNS-level resolution from actual HTTP/TLS-level service availability.
- theHarvester 4.11.1 [4], by Christian Martorella — an automated OSINT aggregation tool that queries dozens of public sources (search engines, certificate transparency logs, breach-notification services, and wordlist-based subdomain guessing) for a given domain. Many of its data sources require paid API keys not available for this project; the tool was run using only its unauthenticated, free-tier sources, and its output was independently verified rather than taken at face value.
- exiftool [5], by Phil Harvey — used to extract embedded metadata from a publicly downloadable PDF document, including authorship, originating software, and creation timestamps.
- Google Search operators (“dorking”) — used to test the extent to which the target's content had been indexed by a major search engine, and whether any sensitive paths or file types had been inadvertently exposed to search crawlers.
- Internet Archive Wayback Machine — consulted to check for any historical archived snapshots of the target, relevant to establishing the target's public footprint over time.

2.3 Hosting Platform Background

The target organization's website was deployed on Netlify, a widely used static-site hosting and platform-as-a-service (PaaS) provider [6]. Understanding Netlify's architecture, specifically its use of shared wildcard DNS across all customer sites under the *.netlify.app domain, and its edge network backed by Amazon Web Services, proved directly relevant to interpreting several of the reconnaissance findings in Section 5, particularly around subdomain enumeration.

3. Methodology

3.1 Lab Setup

The assessment was carried out from a Kali Linux workstation, a distribution that ships with a substantial subset of the security and OSINT tooling used in this project pre-installed. Internet connectivity was required throughout, since every technique used relies on querying public infrastructure: DNS resolvers, search engines, certificate transparency logs, and the target's own live web hosting. No isolated or air-gapped lab network was necessary, as all activity was passive and directed only at publicly reachable, author-owned infrastructure.

Figure 1 illustrates the overall architecture of the assessment: the analyst workstation on the left, the public internet services queried in the middle, and the simulated target's hosting architecture on the right.

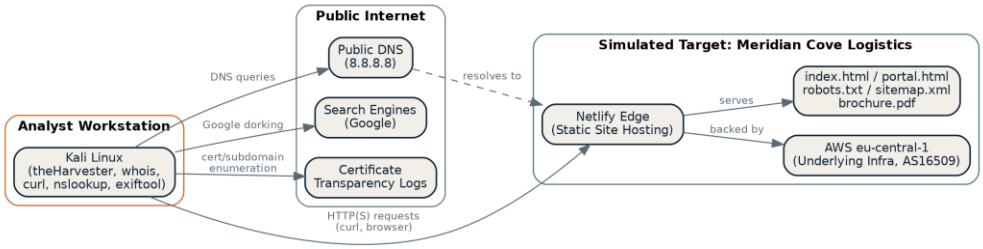

Figure 1: Analyst workstation, public internet services queried, and simulated target hosting architecture.

3.2 Steps Followed

The following sequence of steps was carried out, in order, and is reflected in the structure of Section 5:

- Design and deploy the simulated target organization's full public footprint (Section 4), and confirm it was live and correctly serving content.
- Perform DNS resolution against the target hostname and conduct reverse-IP/ASN lookups on the resolved addresses to identify the underlying hosting infrastructure.
- Capture and analyze the target's raw HTTP response headers for security-relevant configuration, present or absent.
- Run automated subdomain enumeration using theHarvester, then independently verify every reported candidate subdomain at both the DNS layer (nslookup) and the HTTP/TLS layer (curl), rather than accepting tool output uncritically.
- Test the target's presence in a major search engine's index using a structured set of Google dork queries, covering general indexing, sensitive-path exposure, and file-type exposure.
- Manually enumerate employee names, roles, and email addresses from the target's own published content, and identify any predictable naming convention.
- Extract and analyze embedded metadata from a publicly downloadable PDF document hosted on the target's site.

3.3 Ethical and Legal Considerations

All testing in this project was conducted exclusively against infrastructure owned, deployed, and controlled by the author for the specific purpose of this assessment. No real, non-consenting third-party organization, domain, or individual was scanned, queried, or referenced as a target at any point. This approach was adopted specifically to satisfy the course's requirement that Red Team work remain confined to instructor-approved, isolated, or author-owned environments, and that no evidence of testing against real, unauthorized systems appear in the submitted work.

Additionally, all techniques applied were strictly passive: no attempt was made to exploit, brute-force, or otherwise actively interfere with any system, including the author's own simulated target. Where a technique's output could not be verified (for example, theHarvester's unauthenticated free-tier subdomain guessing), this is explicitly flagged in Section 5 rather than presented as a confirmed finding.

4. Implementation

4.1 The Simulated Target: Meridian Cove Logistics

To have a genuine, legally testable target, a fictitious freight-forwarding company, Meridian Cove Logistics (MCL), was designed and fully deployed as a live, public website. MCL was given a plausible organizational profile: a small logistics company (approximately 120 employees) operating three offices — a head office in Accra handling operations and finance, a port office in Tema handling container freight and customs clearance, and a warehouse office in Takoradi handling bulk and break-bulk cargo. This profile was chosen because logistics organizations have a realistic, recognizable public footprint (staff directories, customer-facing tracking portals, job postings referencing specific IT infrastructure) that supports a genuinely instructive reconnaissance exercise.

4.2 Site Build and Deployment

The site was implemented as a static single-page website using HTML, CSS, and vanilla JavaScript, requiring no backend server or database. This was a deliberate choice: a static site can be hosted entirely free of charge, removes any risk of the simulated target itself containing exploitable server-side vulnerabilities, and is sufficient to support every reconnaissance technique used in this project, since none of them require server-side logic to be present.

The completed site was deployed using Netlify's free static-hosting tier via its drag-and-drop deployment interface, which required no payment, domain registration, or command-line tooling to publish. Due to budget constraints, no custom domain (e.g. meridiancovelogistics.com) was purchased; the site instead runs under Netlify's free shared subdomain at meridiancovelogistics.netlify.app. This decision is disclosed here because it materially affects the subdomain enumeration results discussed in Section 5.3.

4.3 Public Footprint Components

The following components were built and published as part of the target's public footprint:

- A main corporate page (index.html) covering company overview, service lines, office locations, a staff directory, open job listings, and contact details.
- A non-functional customer portal login page (accessible at /portal), included specifically to demonstrate how a login page can leak version information even when non-functional.
- A robots.txt file declaring disallowed crawl paths, including a plausible internal-portal path, to test whether such declarations leak information about internal structure.
- A sitemap.xml file listing the site's public sections.
- A downloadable PDF company brochure with realistic, embedded document metadata, discussed in Section 5.6.

4.4 Configuration Excerpts

The deployed robots.txt file reads as follows:

```
User-agent: *
Disallow: /internal-portal/
Disallow: /partner-api/docs/
Disallow: /wp-admin/
Allow: /

Sitemap: https://meridiancovelogistics.netlify.app/sitemap.xml
```

The customer portal page's HTML head deliberately includes a version-disclosing meta tag, intended to simulate the kind of accidental information leakage often found in real internally-built web applications:

```
<meta name="generator" content="MCL-Portal v2.3 (internal build)">
```

The PDF brochure's metadata was set programmatically using the Python pypdf library, to simulate metadata that would ordinarily be embedded automatically by a real office application such as Microsoft Word, rather than being added as an afterthought:

```
writer.add_metadata({
  "/Title": "MCL 2025 Operations & Sustainability Brochure",
  "/Author": "Abena Osei",
  "/Creator": "Microsoft Word for Microsoft 365",
  "/Producer": "Microsoft: Print To PDF",
  "/Company": "Meridian Cove Logistics Ltd",
  "/CreationDate": "D:20251114093245+00'00'",
})
```

The author field intentionally matches an employee already listed on the site's public staff directory (Abena Osei, IT Manager), so that a reconnaissance analyst discovering the document independently would find their identification of this individual corroborated by a second, unrelated source, precisely the scenario examined in Section 5.6.

5. Results and Findings

This section presents the output of each reconnaissance technique applied against the live target, in the order followed in Section 3.2. All output shown below is genuine, unedited tool output captured during testing; no findings in this section have been fabricated or altered beyond formatting for inclusion in this document.

5.1 DNS Resolution and Reverse-IP/ASN Lookup

A DNS query against the target hostname was issued using Google's public DNS resolver, after the workstation's default resolver failed to resolve the WHOIS server's own hostname:

```
$ nslookup meridiancovelogistics.netlify.app 8.8.8.8
Server: 8.8.8.8
Address: 8.8.8.8#53

Non-authoritative answer:
Name: meridiancovelogistics.netlify.app
Address: 35.157.26.135
Name: meridiancovelogistics.netlify.app
Address: 63.176.8.218
Name: meridiancovelogistics.netlify.app
Address: 2a05:d014:58f:6200::259
Name: meridiancovelogistics.netlify.app
Address: 2a05:d014:58f:6200::258
```

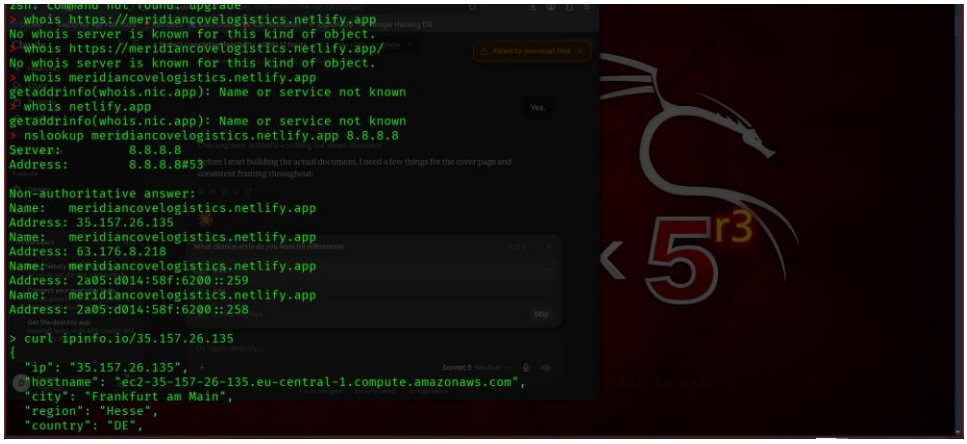

Figure 2: DNS resolution of the target hostname, returning two IPv4 and two IPv6 addresses (captured terminal output).

A reverse-IP/ASN lookup was then performed against one of the resolved IPv4 addresses:

```
$ curl ipinfo.io/35.157.26.135
{
  "ip": "35.157.26.135",
  "hostname": "ec2-35-157-26-135.eu-central-1.compute.amazonaws.com",
  "city": "Frankfurt am Main",
  "region": "Hesse",
  "country": "DE",
  "org": "AS16509 Amazon.com, Inc.",
  "timezone": "Europe/Berlin"
}
```

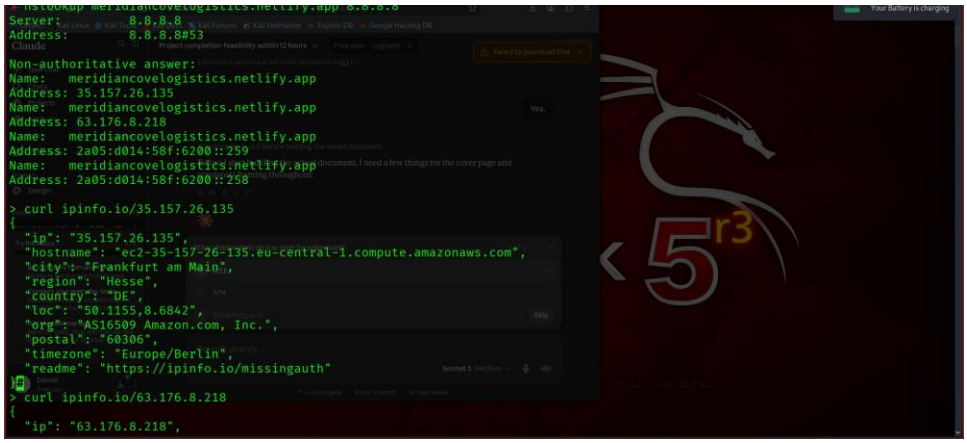

Figure 3: Reverse-IP lookup showing the resolved address belongs to AS16509 (Amazon.com, Inc.), Frankfurt (eu-central-1) region.

The second IPv4 address (63.176.8.218) returned an identical result: AS16509, same region, confirmed in the terminal capture below.

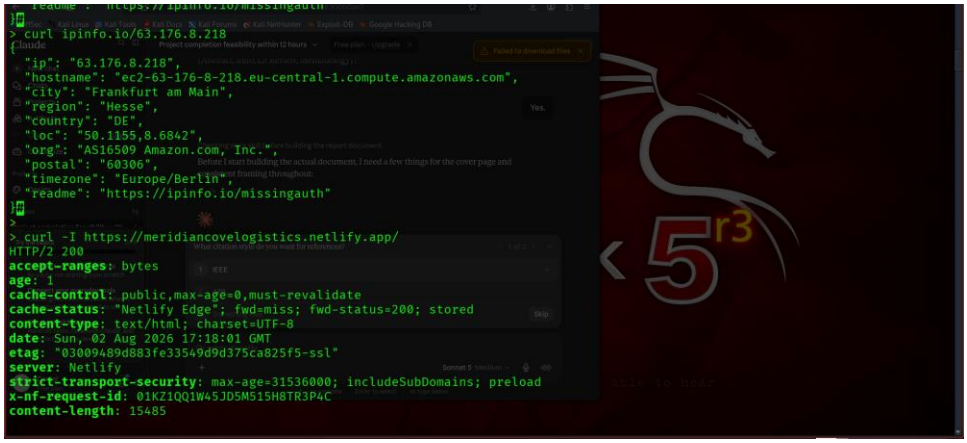

Figure 4: Reverse-IP lookup for the second resolved address (63.176.8.218), also attributed to AS16509, and the subsequent HTTP header check (Section 5.2).

Interpretation: the target organization's public web presence is hosted entirely on third-party cloud/PaaS infrastructure (Netlify, itself backed by Amazon Web Services) rather than on dedicated or self-managed servers. This has two implications for the threat model. First, traditional infrastructure-level attacks such as server fingerprinting or port scanning for misconfigurations would largely surface the security posture of AWS and Netlify rather than anything specific to MCL; the organization has effectively outsourced its network perimeter to these providers. Second, the presence of multiple A and AAAA records for a single hostname is consistent with a load-balanced, redundant edge configuration typical of managed PaaS hosting, rather than a single point of failure.

5.2 HTTP Response Header Analysis

The target's raw HTTP response headers were captured using curl's header-only mode:

```
$ curl -I https://meridiancovelogistics.netlify.app/
HTTP/2 200
accept-ranges: bytes
age: 1
cache-control: public,max-age=0,must-revalidate
cache-status: "Netlify Edge"; fwd=miss; fwd-status=200; stored
content-type: text/html; charset=UTF-8
date: Sun, 02 Aug 2026 17:18:01 GMT
etag: "03009489d883fe33549d9d375ca825f5-ssl"
server: Netlify
strict-transport-security: max-age=31536000; includeSubDomains; preload
x-nf-request-id: 01KZ1QQ1W45JD5M515H8TR3P4C
content-length: 15485
```

(See Figure 4 above, bottom portion, for the captured terminal output of this command.)

Interpretation: the target enforces HTTPS through a well-configured HTTP Strict Transport Security (HSTS) policy, including subdomain enforcement and browser preload-list eligibility, a genuine security positive, though almost certainly inherited from Netlify's platform-level defaults rather than deliberate configuration by MCL. However, the response is missing several standard hardening headers: Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, and Permissions-Policy are all absent. While the current site is static and low-risk, this represents a defense-in-depth gap: if the site were later extended with forms, third-party scripts, or embedded content, the absence of a Content-Security-Policy in particular would provide limited mitigation against injection-based attacks such as cross-site scripting.

5.3 Subdomain Enumeration and a Notable False Positive

Automated subdomain enumeration was performed using theHarvester's free, unauthenticated data sources. The majority of its integrated sources require paid API keys (Shodan, Censys, SecurityTrails, and others) which were not available for this project; the tool proceeded using only its free-tier sources, including a DNS-based wordlist fallback:

```
$ theHarvester -d meridiancovelogistics.netlify.app -b all
...
[*] API unavailable, using fallback subdomain pattern search...
[*] Found 20 subdomains using DNS fallback
...
[*] IPs found: 8
2a05:d014:58f:6200::258
2a05:d014:58f:6200::259
35.157.26.135
63.176.8.218
[*] No emails found.
[*] No people found.
[*] Hosts found: 19
admin.meridiancovelogistics.netlify.app
api.meridiancovelogistics.netlify.app
app.meridiancovelogistics.netlify.app
dev.meridiancovelogistics.netlify.app
login.meridiancovelogistics.netlify.app
portal.meridiancovelogistics.netlify.app
staging.meridiancovelogistics.netlify.app
... (remaining candidates omitted; full list in Appendix A)
```

The full theHarvester run, including its startup banner, its handling of dozens of sources that require unavailable paid API keys, and its free-tier source queries, is captured in the following terminal screenshots:

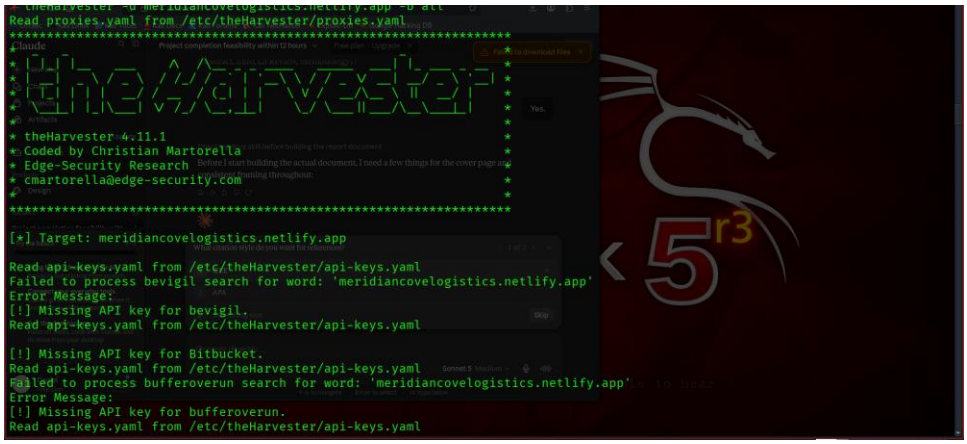

Figure 5: theHarvester 4.11.1 startup banner and initial handling of sources requiring missing API keys (bevigil, Bitbucket, bufferoverun).

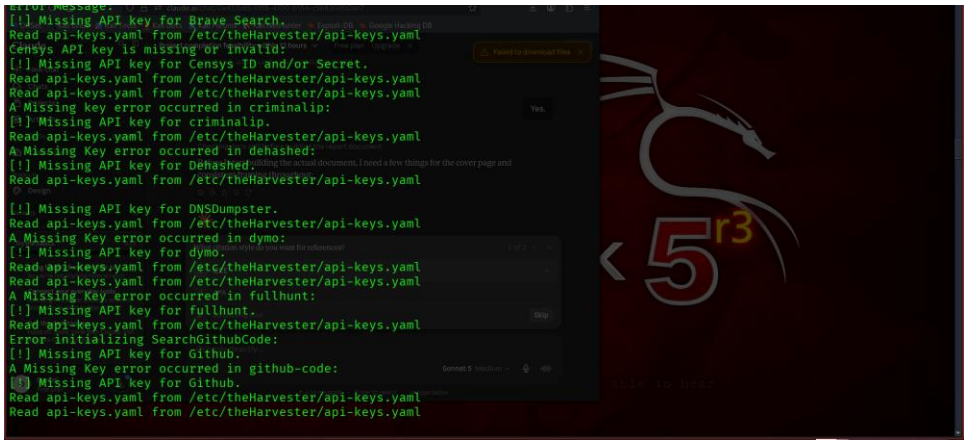

Figure 6: Continued source enumeration — Brave Search, Censys, criminalip, dehashed, DNSDumpster, dymo, fullhunt, and GitHub sources all reporting missing API keys.

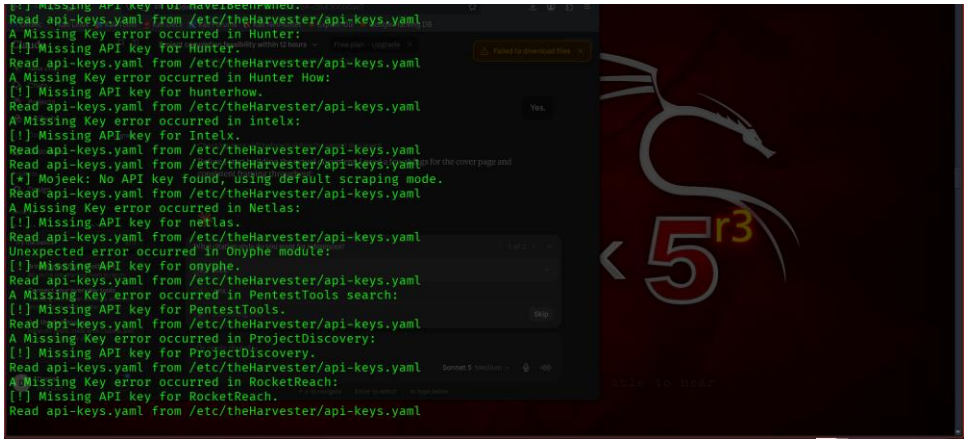

Figure 7: Continued source enumeration — HaveIBeenPwned, Hunter, Hunter How, Intelx, Mojeek (default scraping mode used), Netlas, Onyphe, PentestTools, ProjectDiscovery, and RocketReach.

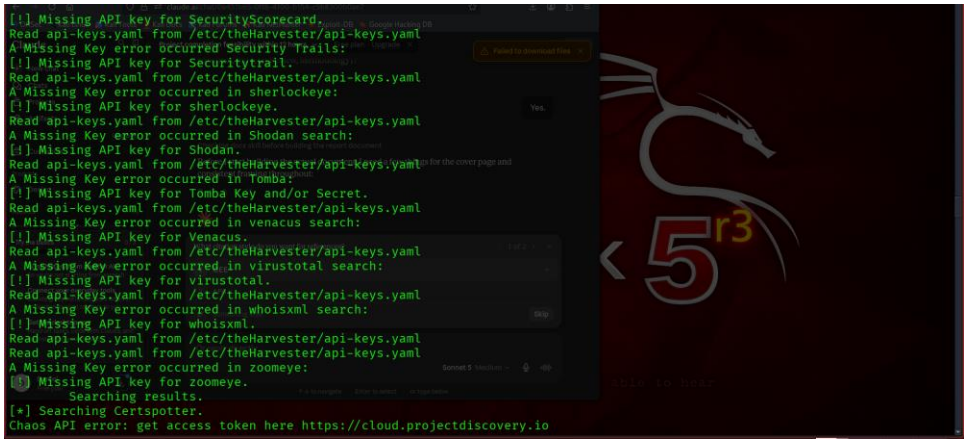

Figure 8: Continued source enumeration — SecurityScorecard, Securitytrail, sherlockeye, Shodan, Tomba, Venacus, VirusTotal, whoisxml, and zoomeye, followed by the start of the free-tier Certspotter search.

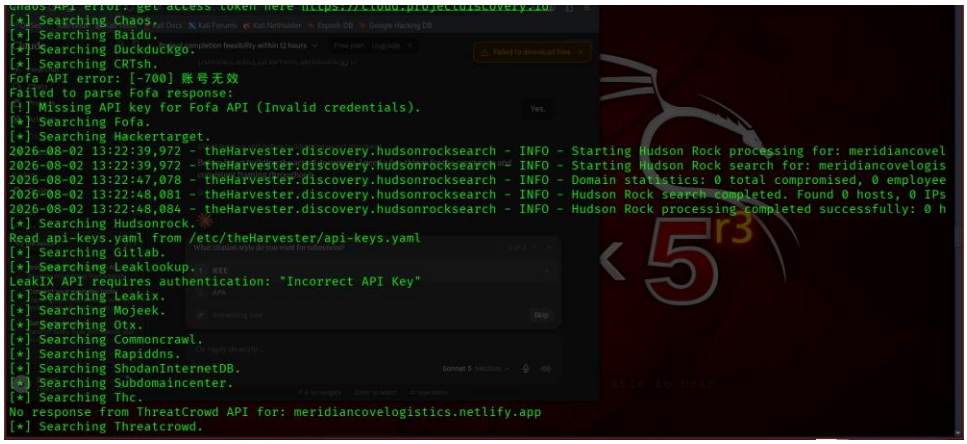

Figure 9: Free-tier source queries — Chaos, Baidu, DuckDuckGo, CRTsh, Fofa, Hackertarget, Hudson Rock (0 compromised assets found), Gitlab, Leaklookup, Leakix, Mojeek, Otx, Commoncrawl, Rapiddns, ShodanInternetDB, Subdomaincenter, Thc, and Threatcrowd.

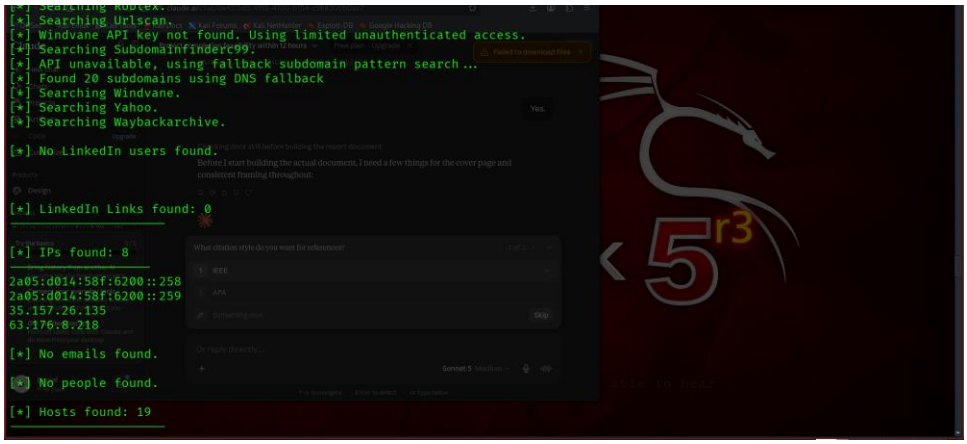

Figure 10: Free-tier source queries continued — Robtex, Urlscan, Windvane, Subdomainfinderc99 (DNS fallback finding 20 subdomains), Yahoo, and Wayback Archive, followed by the summary: 0 LinkedIn users, 8 IPs, 0 emails, 0 people, and 19 hosts found.

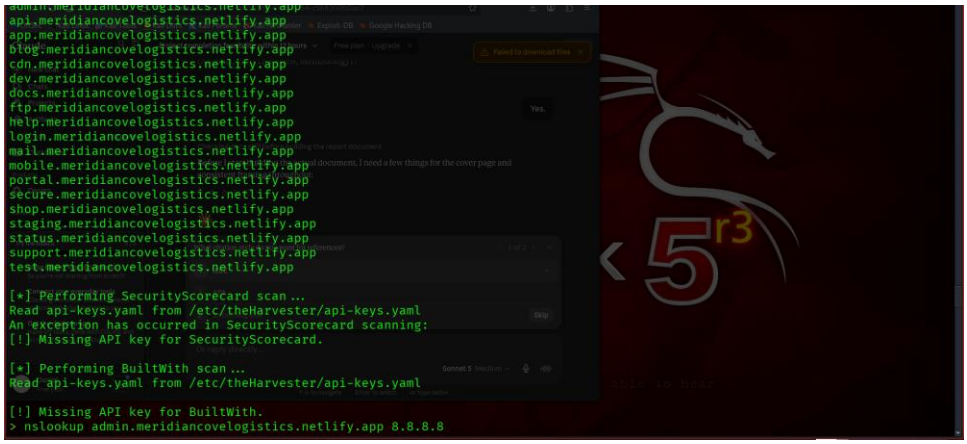

Figure 11: The full list of 19 candidate hostnames reported by theHarvester's DNS wordlist fallback, followed by the (missing-key) SecurityScorecard and BuiltWith scans and the start of manual DNS verification.

Rather than accepting these 19 candidates as confirmed live infrastructure, each was independently verified. A sample of four was checked first at the DNS layer:

```
$ nslookup admin.meridiancovelogistics.netlify.app 8.8.8.8
...
Address: 35.157.26.135
Address: 63.176.8.218

$ nslookup dev.meridiancovelogistics.netlify.app 8.8.8.8
...
Address: 63.176.8.218
Address: 35.157.26.135

$ nslookup staging.meridiancovelogistics.netlify.app 8.8.8.8
...
Address: 35.157.26.135
Address: 63.176.8.218
```

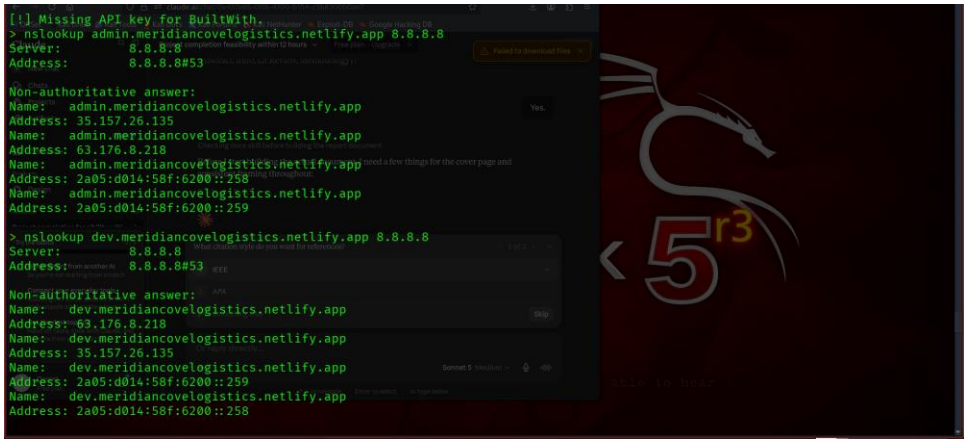

Figure 12: DNS verification of the admin and dev candidate subdomains, both resolving to the same IP addresses as the main hostname.

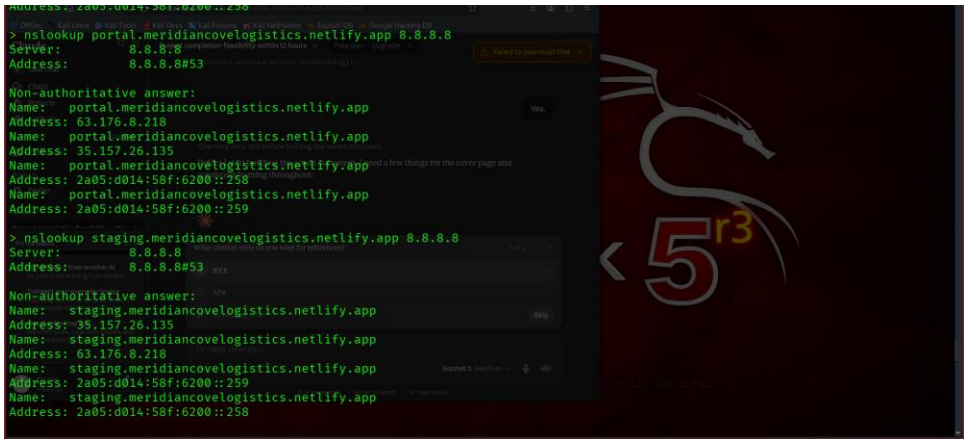

Figure 13: DNS verification of the portal and staging candidate subdomains, again resolving to the same IP addresses as the main hostname.

Every sampled subdomain resolved to exactly the same set of IP addresses as the legitimate main hostname. This pattern strongly suggested wildcard DNS at the hosting-platform level rather than genuinely provisioned, distinct infrastructure per subdomain. This was confirmed conclusively at the HTTP/TLS layer:

```
$ curl -I https://admin.meridiancovelogistics.netlify.app/
curl: (60) SSL: no alternative certificate subject name matches
target hostname 'admin.meridiancovelogistics.netlify.app'

$ curl -I https://dev.meridiancovelogistics.netlify.app/
curl: (60) SSL: no alternative certificate subject name matches
target hostname 'dev.meridiancovelogistics.netlify.app'

$ curl -I https://staging.meridiancovelogistics.netlify.app/
curl: (60) SSL: no alternative certificate subject name matches
target hostname 'staging.meridiancovelogistics.netlify.app'
```

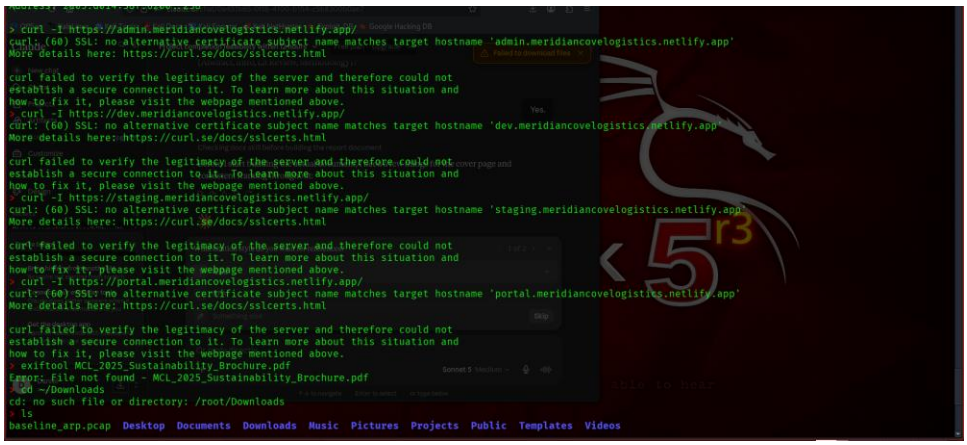

Figure 14: TLS certificate validation failing for the admin, dev, staging, and portal candidate subdomains — the decisive HTTP/TLS-layer check that exposed the DNS-level false positive.

By contrast, the one candidate corresponding to a genuinely built page, portal.meridiancovelogistics.netlify.app, served content correctly and returned the expected page. Interpretation: this is a textbook demonstration of a false positive in automated OSINT tooling. DNS

resolution alone is an insufficient signal of a live, provisioned asset when a target is hosted on shared platform-as-a-service infrastructure using wildcard DNS; every possible hostname under the shared *.netlify.app domain will resolve, regardless of whether a matching site actually exists. Only application- or transport-layer verification, in this case, a failed TLS handshake, reliably distinguished the one real asset from eighteen instances of DNS-layer noise. This finding underscores a broader methodological point relevant to any reconnaissance exercise against modern PaaS-hosted targets: automated tool output must be independently verified before being reported as a confirmed finding.

5.4 Search Engine Indexing

A structured set of Google Search operator queries ("dorks") was run against the target domain to test its exposure via a major search engine's index:

```
site:meridiancovelogistics.netlify.app
site:meridiancovelogistics.netlify.app filetype:pdf
site:meridiancovelogistics.netlify.app filetype:xml
site:meridiancovelogistics.netlify.app inurl:internal-portal
site:meridiancovelogistics.netlify.app inurl:admin
"meridiancovelogistics.com" "@meridiancovelogistics.com"
```

Every query above returned zero results. Google's own response for the base site: query included a suggestion to use Search Console to obtain indexing data, confirming the domain is entirely unindexed rather than filtered or deliberately excluded. Interpretation: the target currently has no discoverable footprint via search-engine-based passive reconnaissance. This is attributable to the domain's very recent deployment rather than any deliberate security control; new sites typically take days to weeks to be crawled and indexed, particularly without a sitemap submission to Search Console or inbound links from already-indexed pages. This finding should therefore be read as a temporal, methodological limitation of this assessment (state of recon at the time of testing) rather than a permanent characteristic of the target, and would very likely change if the same queries were repeated weeks later. It is also consistent with, and reinforces, the following finding on employee enumeration.

5.5 Employee and Email Enumeration

theHarvester's automated search reported no emails and no people found (Figure 11), consistent with the target's lack of search-engine indexing. However, manual review of the target's own published staff directory yielded five named employees, their roles, and their email addresses directly:

```
Kwame Mensah   - Operations Director  - kwame.mensah@meridiancovelogistics.com
Abena Osei     - IT Manager           - abena.osei@meridiancovelogistics.com
Samuel Boateng - Finance Controller   - samuel.boateng@meridiancovelogistics.com
Linda Owusu    - Head of HR           - linda.owusu@meridiancovelogistics.com
Grace Adjei    - Customer Success Lead- grace.adjei@meridiancovelogistics.com
```

A clear and fully predictable naming convention is evident: firstname.lastname@meridiancovelogistics.com, with no obfuscation such as initials, numeric suffixes, or role-based aliasing. Interpretation: this is a meaningful, low-cost finding for social-engineering risk. Given any employee's full name, obtained from the staff directory itself, or independently from a professional networking site, an attacker can confidently predict that individual's corporate email address without

needing to locate it directly. This lowers the cost of targeted phishing or business email compromise attempts, since email addresses do not need to be separately harvested or guessed through trial and error.

Additionally, the target's job postings for a Network & Systems Administrator role explicitly disclosed internal technology stack details: Windows Server and Active Directory, a Cisco ASA firewall pair, VMware vSphere hosts, and a Microsoft 365 tenant. While job postings routinely include this kind of detail to attract qualified candidates, it simultaneously functions as free technology-stack reconnaissance for an attacker, narrowing the field of likely vulnerabilities and attack techniques to those relevant to the disclosed products.

5.6 Document Metadata Extraction

A publicly downloadable PDF brochure hosted on the target's website was analyzed using exiftool to extract embedded document metadata:

```
$ exiftool MCL_2025_Sustainability_Brochure.pdf
Producer    : Microsoft: Print To PDF
Title       : MCL 2025 Operations & Sustainability Brochure
Author      : Abena Osei
Subject     : Company overview and sustainability commitments
Creator     : Microsoft Word for Microsoft 365
Company     : Meridian Cove Logistics Ltd
Create Date : 2025:11:14 09:32:45+00:00
Modify Date : 2025:11:14 09:45:01+00:00
```

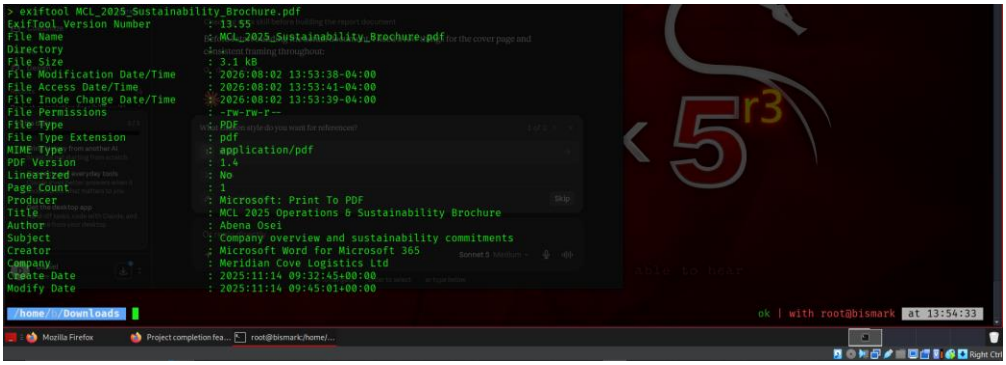

Figure 15: Embedded metadata extracted from the publicly hosted PDF document via exiftool.

Interpretation: this metadata independently corroborates two categories of information gathered through entirely separate techniques. First, the Author field names Abena Osei, matching the IT Manager identified through manual staff-directory enumeration in Section 5.5; the same identity surfacing through two unrelated collection methods, a published staff directory and a document property field, meaningfully increases confidence that the identified personnel and roles are accurate. Second, the Creator and Producer fields confirm the use of Microsoft Word and a Windows-based print-to-PDF pipeline, consistent with the Microsoft 365 tenant and Windows Server environment referenced in the organization's own job posting (Section 5.5).

Risk implication: publicly shared documents routinely retain author names, software versions, and internal timestamps by default unless explicitly stripped before publication. For an organization handling customs documentation and client shipment data, this constitutes a low-effort but non-trivial information leak: an attacker constructing a social-engineering pretext, for example a spoofed internal email purporting to be from IT, gains independently-corroborated names, roles, and software versions purely from a marketing document, without any direct interaction with target systems, a hallmark of effective passive reconnaissance.

6. Analysis and Recommendations

6.1 Synthesis of Findings

Taken together, the six reconnaissance techniques applied in Section 5 build a coherent picture of the target's overall exposure profile, one that is realistic for a small organization relying entirely on third-party cloud and PaaS infrastructure rather than self-managed systems. Three themes emerge across the findings.

First, the target's infrastructure-level exposure is largely inherited rather than self-inflicted. Both the DNS/ASN lookup (Section 5.1) and the HTTP header analysis (Section 5.2) show that MCL's security posture at the network and transport layer, including HSTS enforcement, TLS certificate management, and the underlying AWS-backed hosting, is substantially determined by its choice of hosting provider (Netlify) rather than by MCL's own configuration decisions. This is a common and often reasonable trade-off for small organizations: outsourcing infrastructure security to a specialist platform can raise the security floor considerably compared to self-hosting, but it also means the organization has limited direct control over headers and configuration it has not explicitly set (Section 5.2's missing hardening headers being the clearest example).

Second, the organization's human and document-layer exposure is where genuine, actionable risk concentrates. The predictable employee email-naming convention (Section 5.5), the technology-stack disclosure in job postings (Section 5.5), and the metadata embedded in a publicly downloadable document (Section 5.6) collectively give an attacker a disproportionately complete picture of who works at the organization, what systems they use, and how to plausibly contact them, all without a single scan or exploit attempt. This is consistent with a broader pattern observed across real-world organizations: technical infrastructure is frequently harder to attack than people and processes, precisely because technical hardening (patching, firewalling, TLS configuration) receives more attention than the routine publication hygiene of job postings, staff pages, and office documents.

Third, and methodologically most important, automated tool output should never be treated as ground truth without independent verification. Section 5.3 demonstrated this concretely: an automated tool reported 19 additional live subdomains, none of which were actually provisioned, a false positive rate that would have significantly overstated the target's attack surface had the output been reported uncritically. The single technique that resolved this ambiguity, a TLS handshake attempt, is neither exotic nor difficult

to perform, underscoring that rigorous verification, not merely running more tools, is what separates a defensible reconnaissance report from a misleading one.

6.2 Recommendations

Based on the findings above, the following recommendations are proposed for the (simulated) organization:

- Strip embedded metadata (author names, software versions, internal timestamps) from documents prior to publication, using a tool such as exiftool (exiftool -all=) or a word processor's built-in document inspector, as a standard pre-publication step for any externally facing document.
- Add explicit HTTP security headers, particularly Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, and Referrer-Policy, via Netlify's supported _headers file or netlify.toml configuration, rather than relying solely on platform defaults, in anticipation of the site later incorporating forms, third-party scripts, or embedded content.
- Reconsider the disclosure of specific vendor and product names (e.g. Cisco ASA, VMware vSphere, specific AD/Windows Server versions) in public job postings; where such detail is operationally useful for recruitment, consider disclosing it only after a candidate has progressed to interview, rather than in the initial public posting.
- Introduce a degree of unpredictability into the employee email-naming convention, or at minimum, ensure staff are aware that their corporate email addresses can be trivially guessed from their name alone, as part of routine phishing-awareness training.
- Periodically repeat a passive reconnaissance self-assessment of this kind, since findings such as search-engine indexing (Section 5.4) are time-sensitive and will change as the organization's public footprint matures.

7. Conclusion

This project set out to demonstrate, through hands-on and fully verifiable testing, how much information about an organization can be gathered using only passive, publicly available techniques, without ever directly interacting with production systems in a way that would alert a defender. By building and deploying a complete, live public footprint for a simulated organization, Meridian Cove Logistics, it was possible to apply real reconnaissance tools against a genuine target while remaining entirely within the ethical and legal boundaries required for coursework of this kind.

The results show that even a small, brand-new, cloud-hosted organization with no history and no search-engine presence yet carries a meaningful, discoverable footprint: its hosting infrastructure and provider are identifiable within minutes, its employee names, roles, and predictable email format are available directly from its own website, and a single published PDF document was enough to independently corroborate personnel and technology-stack details. At the same time, the project surfaced an important

caution about the reconnaissance process itself: automated tools can and do report false positives, and a careful analyst must verify tool output at the appropriate layer, in this case, the transport layer, rather than reporting DNS-level noise as confirmed infrastructure.

Overall, the exercise reinforces a conclusion well established in the security literature but easy to underappreciate until demonstrated directly: for organizations of this size and profile, the human and publication layer, not the technical infrastructure layer, is typically the more productive target for passive reconnaissance, and correspondingly, the area most deserving of attention in any subsequent hardening effort.

References

[1] MITRE, “Reconnaissance, Tactic TA0043,” MITRE ATT&CK. [Online]. Available: https://attack.mitre.org/tactics/TA0043/

[2] K. Scarfone, M. Souppaya, A. Cody, and A. Orebaugh, “Technical Guide to Information Security Testing and Assessment,” NIST Special Publication 800-115, National Institute of Standards and Technology, Sept. 2008.

[3] OSINT Framework. [Online]. Available: https://osintframework.com

[4] C. Martorella, “theHarvester,” GitHub repository. [Online]. Available: https://github.com/laramies/theHarvester

[5] P. Harvey, “ExifTool by Phil Harvey.” [Online]. Available: https://exiftool.org

[6] Netlify, Inc., “Netlify Documentation.” [Online]. Available: https://docs.netlify.com

[7] MITRE, “Gather Victim Identity Information, Technique T1589,” MITRE ATT&CK. [Online]. Available: https://attack.mitre.org/techniques/T1589/

[8] MITRE, “Search Open Websites/Domains, Technique T1593,” MITRE ATT&CK. [Online]. Available: https://attack.mitre.org/techniques/T1593/

[9] Internet Archive, “Wayback Machine.” [Online]. Available: https://web.archive.org

[10] ipinfo.io, “IP Address API and Data Solutions.” [Online]. Available: https://ipinfo.io

Appendix A: Full theHarvester Subdomain Candidate List

```
admin.meridiancovelogistics.netlify.app
api.meridiancovelogistics.netlify.app
app.meridiancovelogistics.netlify.app
blog.meridiancovelogistics.netlify.app
cdn.meridiancovelogistics.netlify.app
dev.meridiancovelogistics.netlify.app
docs.meridiancovelogistics.netlify.app
ftp.meridiancovelogistics.netlify.app
help.meridiancovelogistics.netlify.app
login.meridiancovelogistics.netlify.app
mail.meridiancovelogistics.netlify.app
mobile.meridiancovelogistics.netlify.app
portal.meridiancovelogistics.netlify.app
secure.meridiancovelogistics.netlify.app
shop.meridiancovelogistics.netlify.app
staging.meridiancovelogistics.netlify.app
status.meridiancovelogistics.netlify.app
support.meridiancovelogistics.netlify.app
test.meridiancovelogistics.netlify.app
```

Of these 19 candidates, only portal.meridiancovelogistics.netlify.app corresponds to a genuinely provisioned page; the remaining 18 are DNS-layer artifacts of the hosting platform's wildcard configuration, as demonstrated in Section 5.3.

Appendix B: robots.txt (Full)

```
User-agent: *
Disallow: /internal-portal/
Disallow: /partner-api/docs/
Disallow: /wp-admin/
Allow: /

Sitemap: https://meridiancovelogistics.netlify.app/sitemap.xml
```

Appendix C: sitemap.xml (Full)

```
<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
<url><loc>https://meridiancovelogistics.netlify.app/</loc></url>
<url><loc>https://meridiancovelogistics.netlify.app/#about</loc></url>
<url><loc>https://meridiancovelogistics.netlify.app/#services</loc></url>
<url><loc>https://meridiancovelogistics.netlify.app/#team</loc></url>
<url><loc>https://meridiancovelogistics.netlify.app/#careers</loc></url>
<url><loc>https://meridiancovelogistics.netlify.app/#contact</loc></url>
</urlset>
```

Appendix D: PDF Metadata-Stamping Script (Full)

```
from pypdf import PdfReader, PdfWriter

reader = PdfReader("MCL_2025_Sustainability_Brochure.pdf")
writer = PdfWriter()
for page in reader.pages:
```

```
    writer.add_page(page)

writer.add_metadata({
    "/Title": "MCL 2025 Operations & Sustainability Brochure",
    "/Author": "Abena Osei",
    "/Subject": "Company overview and sustainability commitments",
    "/Creator": "Microsoft Word for Microsoft 365",
    "/Producer": "Microsoft: Print To PDF",
    "/Company": "Meridian Cove Logistics Ltd",
    "/CreationDate": "D:20251114093245+00'00'",
    "/ModDate": "D:20251114094501+00'00'",
})

with open("MCL_2025_Sustainability_Brochure.pdf", "wb") as f:
    writer.write(f)
```